CSCI 3308 Final Project

- **Team Number:** 013-4
- **Team Name:** Send It!
- **Team Members:**
    - Brock Benton (brockbenton, brock.benton@colorado.edu)
    - Sam Kasten (killthecreep, sam.kasten@colorado.edu)
    - Atharva Joshi (Joshath25, atharva.joshi@colorado.edu)
    - Alex Jost (aljostbailey, aljo3097@colorado.edu)
    - Ganesh Akhil Meduri (Akhil986, game7773@colorado.edu)

SpotDrop

Description: A community-driven map and social platform where urban athletes discover, share, and pin the best spots for climbing, skating, skiing, parkour, and more. This is complete with geo-tagged locations, photos, videos, difficulty ratings, and reviews filtered by sport type and proximity. The app includes a built-in forum where users can connect with friends and save spots to personalized collections for future sessions.

Audience: Climbers, Skiers, Skaters, and everyone else that enjoys outdoor activities and sports.

Vision Statement: To become the go-to platform where urban sport communities worldwide collaborate to map the uncharted playground hidden in every city.

Development Methodology

SpotDrop employs a Scrum-Kanban hybrid approach optimized for a compressed 4-week MVP development cycle. This methodology combines the structure of Scrum sprints with the continuous flow of Kanban, enabling rapid iteration while maintaining clear milestones and accountability.

The project is organized into four 1-week sprints, each representing a distinct development phase:

- Week 1: Database schema design, API endpoint setup, and map integration
- Week 2: Geo-tagging functionality, media upload capability, and user authentication

- Week 3: Community forum integration, spot filtering implementation, and UI/UX polish
- Week 4: Bug hunting, user testing, pitch preparation, and MVP deployment

Development progress is tracked on a GitHub Project Board using a table view with the following columns:

- Title – Issue name and ticket number
- Assignees – Team member responsible for the task
- Status – Current stage (Todo → In Progress → Done)
- Linked Pull Requests – Associated code changes for traceability
- Sub-Issues Progress – Real-time completion tracking for epic parent tasks

Issue Hierarchy

- Epics (Parent Issues): 4 weekly development phases (#1-#4)
- User Stories (Sub-Issues): 13 granular, actionable tasks (#5-#17) with defined acceptance criteria

Team members post asynchronous weekly updates in a dedicated discussion thread, answering:

1. What did I accomplish this week?
2. What will I work on next week?
3. What blockers am I facing?

Communication Plan

- We will use Discord for communication. Our channel is separated into two primary channels: “General” and “Project_Workflow.” Our “General” channel is simple communication - when we are meeting, quick questions, and such - while our “Project_Workflow” channel is directed towards technical communication and decision making.

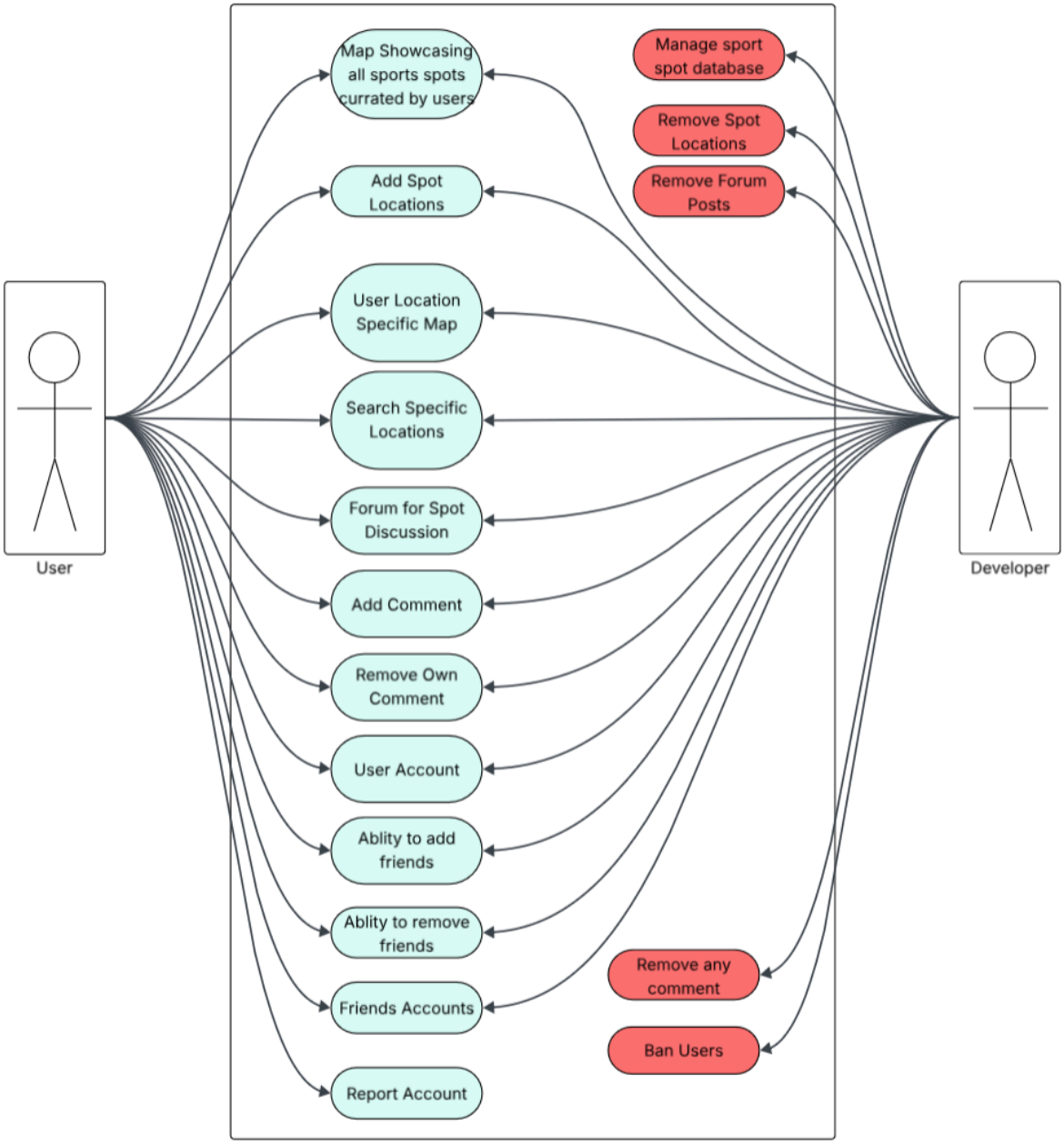
Meeting Plan

- TA Meeting: 3:30-3:45, Weekly on Tuesday, Online via Google Meets
- Team Meeting: 3:45-4:30, Weekly on Tuesday, Online via Google Meets

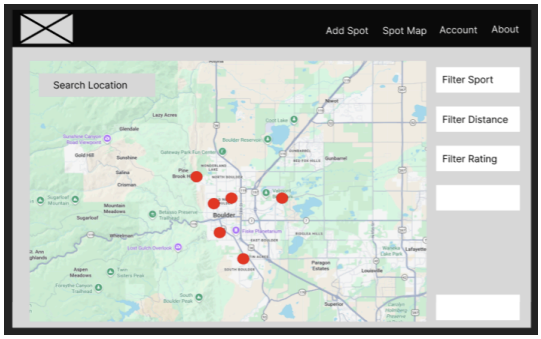
Use Case Diagrams (need 6 key features)

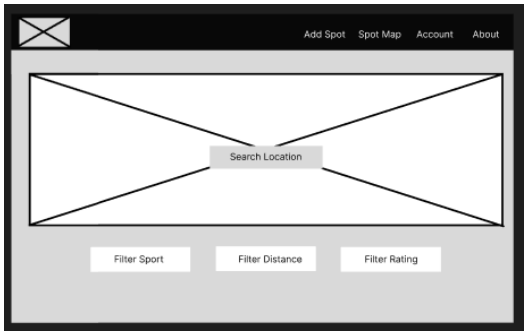
1. Map showcasing the sports location the users have posted in the forum.

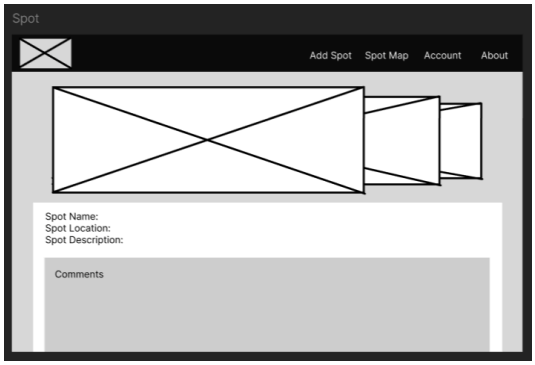
2. Geolocating button to find the users current location and show relevant spots.
3. Search feature to find specific areas.
4. Forums where users can discuss different spots.
5. Log-in functionality to showcase spots submitted or bookmarked.
6. Ability to add friends and see when they post new spots / comment on forum threads.

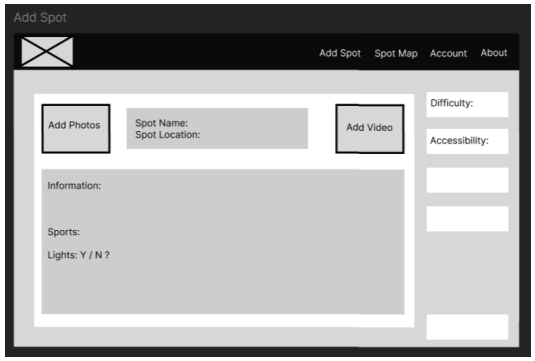


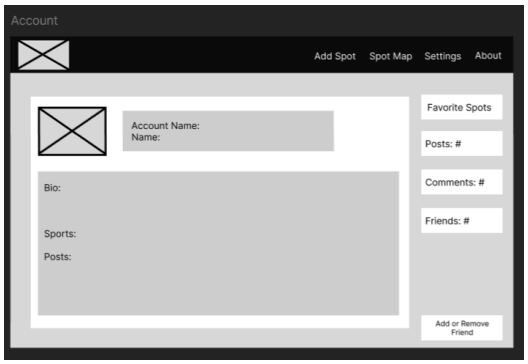
Wireframes

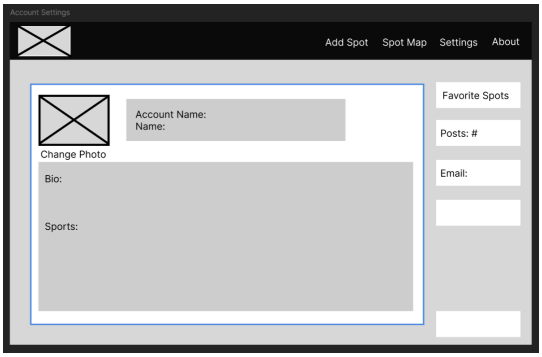












Extra Credit: Potential Issues

1. The forum will live and die by the hands of the users. This means we need to obtain and retain a substantial community to ensure that they are uploading spots and locations. As this is the entire concept of the design, without users the site will not be as effective. This is a pretty severe risk, but we think we can combat this by having a strong foundation.
    a. We can combat this by adding known spots and locations currently being uploaded and used by other social media such as Instagram and Tik-Tok.

Having this database of spots and locations will be helpful to create a strong foundation that users will want to interact with.

2. We need to ensure that the UI is clear and concise. If the UI is clunky and noninformative the users will not use the website to the max potential, if they decide to use the site at all. This is a pretty serious risk, but as long as we start off knowing this risk we can start off on the right foot with some nice UI.
    a. To combat this and ensure our UI is clear and concise we will ensure to do adequate user testing. This will require getting user feedback to ensure, and not only get it, but to take it to heart and see what works and what doesn't, then test it again.

3. We may need to limit the number of spots and locations a user can upload within the same day. This would be to ensure that the server doesn't get overloaded or bogged down with requests by more malicious users. This has a potential for severe risk as we are working with a google API that we are not familiar with so it is better to air on the side of caution.
    a. We will need to do some testing to see what works best, but we should limit users to uploading only a few spots per hour seems like the best idea. Though we need to do some real world tests, such as going to locations and seeing how many spots one could potentially want to upload within a set amount of time. The other option is limiting the user to only a certain number of spot/location uploads per day, again this will take testing to verify what works best.

4. People uploading false or misleading information, but not only that, but we could have a spot that has been destroyed or changed to the point it is unusable. This is relatively severe as if the site is full of misleading information no one will ever use it and once one person gets burned on a spot/location they may never use the site again.
    a. This can be combated by allowing users to report spots/locations that contain false or misleading information, allowing them to state the reason as to why they reported it, something like it was not in the right location or the spot no longer exists. This ensures the community can police itself without much intervention from the Developers.

5. People falsely flagging spots/locations for malicious reasons. This is a pretty severe risk because it will cause users to lose confidence in the product, not because the product is bad, but because one person wanted to cause issues.

a. This will be an interesting issue that we will want to plan for and design around as we do want the users to possibly be able to report misinformation, but we don't want people to abuse that power. We can do this by tracking the number of pages reported by a user and tracking that number, if the number is too high and we can flag that user. Once the user has been flagged we could potentially send an email out to a developer. Once that email has been received a developer can verify the pages reported by the user. The developer can then choose to possibly ban the user or simply remove their ability to report pages for a set amount of time. This will require developers to be able to interact with the profiles within the saved database. This will require research to ensure we do it correctly and additionally we will need to design our databases around the idea the developers may need to interact with it. This will be the most complex risk that us as developers